

Generalized Wasserstein Deep Forest: An Interpretable Deep Model for Imbalanced Pattern Classification

by

Tanujit Chakraborty

Ph.D from Indian Statistical Institute, Kolkata, India

Postdoctoral Fellow, IIIT Delhi, India.



Outline of the talk

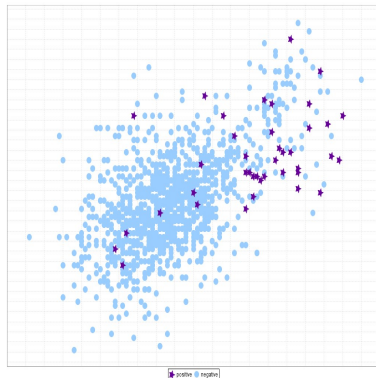
- Imbalanced Classification
- Hellinger Net Model
- Real-life Applications
- Crisis and Future Challenges
- Research Plan: Generalized Wasserstein Deep Forest

Some Parts of the Talk is taken from:

Tanujit Chakraborty and Ashis Chakraborty. "Hellinger Net: A Hybrid Imbalance Learning Model to Improve Software Defect Prediction". **IEEE Transactions on Reliability**, 2020. ([Read Online](#))

Imbalanced Classification Problem

- Real-world data sets are usually skewed, in that many cases belong a larger class and fewer cases belong to a smaller yet usually more exciting class
- For example, consider a binary classification problem with the class distribution of 90 : 10. In this case, a straightforward method of guessing all instances to be positive class would achieve an accuracy of 90%.
- Learning from an imbalanced data set presents a tricky problem in which traditional learning algorithms perform poorly.
- Traditional classifiers usually aim to optimize the overall accuracy without considering the relative distribution of each class.



- One way to deal with the imbalanced data problems is to modify the class distributions in the training data by applying sampling techniques to the data set
- Sampling technique either oversamples the minority class to match the size of the majority class or undersamples the majority class to match the size of the minority class.
- Synthetic minority oversampling technique (SMOTE) is among the most popular methods that oversamples the minority class by generating artificially interpolated data (Chawla et al., 2002, JAIR).
- TL (Tomek links) and ENN (edited nearest neighbor) are popular undersampling approaches (Batista et al., 2004, ACM SIGKDD).
- But these approaches have apparent deficiencies, such as undersampling majority instances may lose potentially useful information of the data set and oversampling increases the size of the training data set which may increase computational cost.
- To overcome these problems, “imbalanced data-oriented” algorithms are designed which can handle class imbalance without any modification to class distribution.

Effect of Class Imbalance on Decision Tree

Let X be attribute and Y be the response class. Here Y^+ denotes majority class, Y^- denotes minority class and n is the total number of instances. Also, let $X^{\geq} \rightarrow Y^+$ and $X^{<} \rightarrow Y^-$ be two rules generated by Classification Tree (CT). Table below shows the number of instances based on the rules created using CT.

Table: An example of notions of classification rules

class and attribute	$X^{\geq}$	$X^{<}$	sum of instances
Y^+	a	b	$a + b$
Y^-	c	d	$c + d$
sum of attributes	$a + c$	$b + d$	n

In the case of imbalanced data set the majority class is always much larger than the size of the minority class and thus we will always have $a + b \gg c + d$. It is clear that the generation of rules based on confidence in CT is biased towards majority class.

Various measures, like information gain (IG), gini index (GI) and misclassification impurity (MI) expressed as a function of confidence, are used to decide which variable to split in the important feature selection stage, get affected by class imbalance.

Effect of Class Imbalance on Distance Measures

Table: An example of notions of classification rules

class and attribute	$X^{\geq}$	$X^{<}$	sum of instances
Y^+	a	b	$a + b$
Y^-	c	d	$c + d$
sum of attributes	$a + c$	$b + d$	n

Using Table, we compute the following:

$$P(Y^+/X^{\geq}) = \frac{a}{a + c} = \text{Confidence}(X^{\geq} \rightarrow Y^+)$$

For an imbalanced data set, Y^+ will occur more frequently with $X^{\geq}$ & $X^{<}$ than to Y^- . So the concept of confidence is a fatal error in an imbalanced classification problem.

Entropy at node t is defined as:

$$\text{Entropy}(t) = - \sum_{j=1,2} P(j/t) \log(P(j/t))$$

Effect of Class Imbalance on Distance Measures

In binary classification, information gain for splitting a node t is defined as:

$$IG = \text{Entropy}(t) - \sum_{i=1,2} \frac{n_i}{n} \text{Entropy}(i) \quad (0.1)$$

where i represents one of the sub-nodes after splitting (assuming we have two sub nodes only), n_i is the number of instances in sub-node i and n is the total number of instances. The objective of classification using CT is to maximize IG which reduces to:

$$\text{Maximize} \left\{ - \sum_{i=1,2} \frac{n_i}{n} \text{Entropy}(i) \right\} \quad (0.2)$$

The maximization problem in eqn. (1.7) reduces to:

$$\begin{aligned} \text{Maximize} \left\{ \frac{n_1}{n} \left[P(Y^+/X \geq) \log(P(Y^+/X \geq)) + P(Y^-/X \geq) \log(P(Y^-/X \geq)) \right] \right. \\ \left. + \frac{n_2}{n} \left[P(Y^+/X <) \log(P(Y^+/X <)) + P(Y^-/X <) \log(P(Y^-/X <)) \right] \right\} \quad (0.3) \end{aligned}$$

The task of selecting the “best” set of features for node i are carried out by picking up the feature with maximum IG. As $P(Y^+/X \geq) \gg P(Y^-/X \geq)$, we face a problem while maximizing eqn. (0.3).

Hellinger Distance

Let (Θ, λ) denote a measurable space. Let us suppose that P and Q be two continuous distributions with respect to the parameter λ having the densities p and q in a continuous space Ω , respectively. Define HD as follows:

$$d_H(P, Q) = \sqrt{\int_{\Omega} (\sqrt{p} - \sqrt{q})^2 d\lambda} = \sqrt{2 \left(1 - \int_{\Omega} \sqrt{pq} d\lambda \right)}$$

where $\int_{\Omega} \sqrt{pq} d\lambda$ is the Hellinger integral. It is noted that HD doesn't depend on the choice of the parameter λ .

For the application of HD as a decision tree criterion, the final formulation can be written as follows:

$$HD = d_H(X_+, X_-) = \sqrt{\sum_{j=1}^k \left(\sqrt{\frac{|X_{+j}|}{|X_+|}} - \sqrt{\frac{|X_{-j}|}{|X_-|}} \right)^2}, \quad (0.4)$$

where $|X_+|$ indicates the number of examples that belong to the majority class in training set and $|X_{+j}|$ is the subset of training set with the majority class and the value j for the feature X . The bigger the value of HD, the better is the discrimination between the features ([Hellinger Distance Decision Tree](#), Chawla et al. 2008, ECML).

Hellinger Net : Basic idea

- Hellinger Net is composed of three basic steps:
 - (a) Converting a DT into rules (HD is used as criterion);
 - (b) Constructing a two hidden layered NN from the rules;
 - (c) Training the MLP using gradient descent backpropagation (Rumelhart, Hinton (1988)).
- In decision trees, the overfitting occurs when the size of the tree is too large compared to the number of training data.
- Instead of using pruning methods (removing child nodes), HN employs a backpropagation NN to give weights to nodes according to their significance.

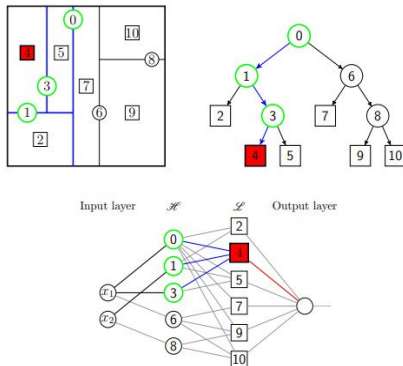
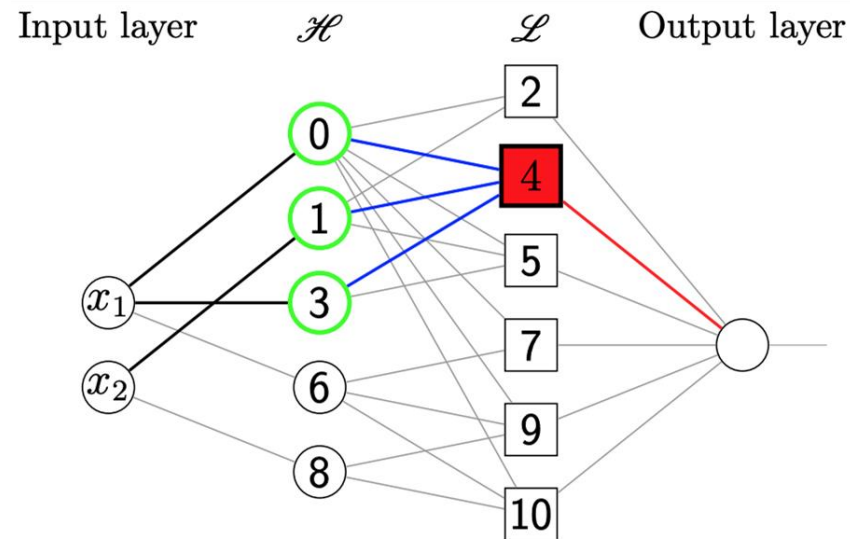
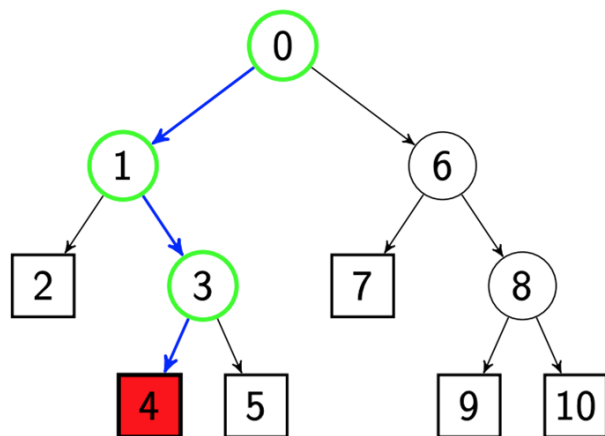


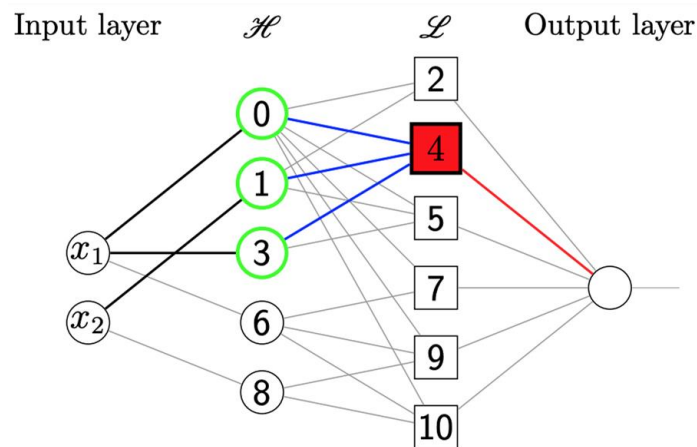
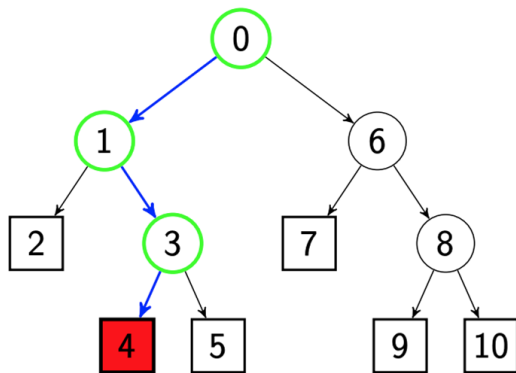
Fig: Graphical Representation of Hellinger Nets

From HDDT to Hellinger Net



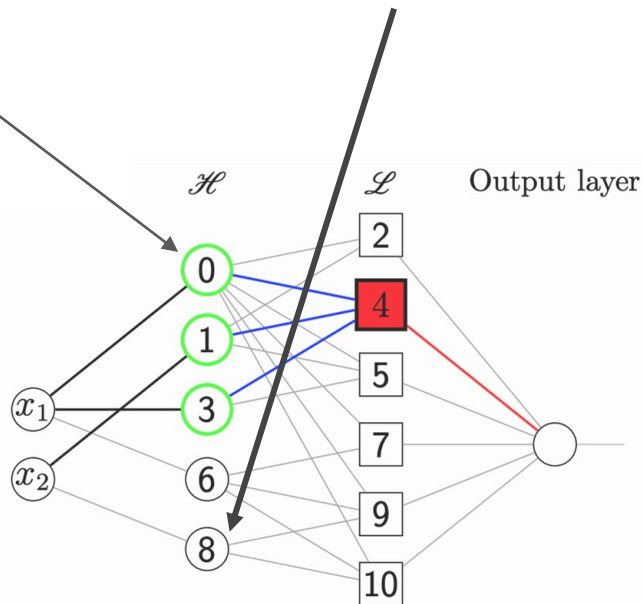
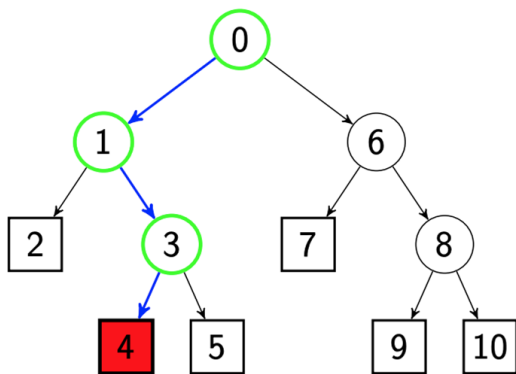
From HDDT to Hellinger Net: Input & First Hidden Layer

- The input layer supplies the features (labeled $\mathbf{x}_i$ or $\mathbf{x}^{(i)}$) to HL1, which corresponds to $k-1$ perceptrons (since our HDDT is a full binary tree, it has $k-1$ internal nodes, and k leaves)
- The first hidden layer (HL1) consists of hyperplanes $H_{k'} = \{x \in C^p : h_{k'}(x) \geq 0\}$ with $h_{k'}(x) = x^{(i_{k'})} - \alpha_{i_{k'}}$ and we're trying to find the side on which $\mathbf{x}$ falls
- The threshold activation function is applied $\tau(u) = 2 \times I_{u \geq 0} - 1$



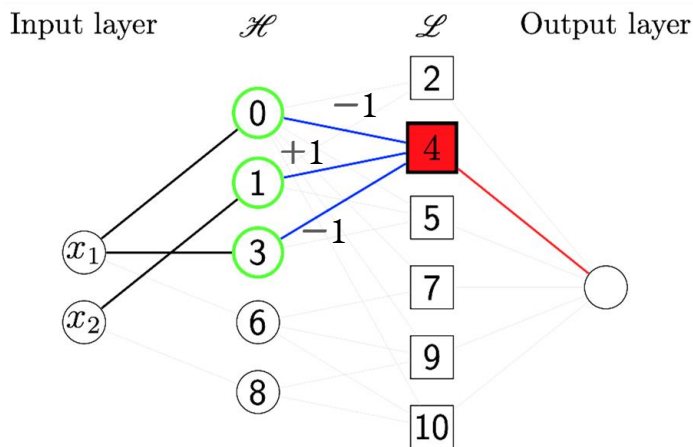
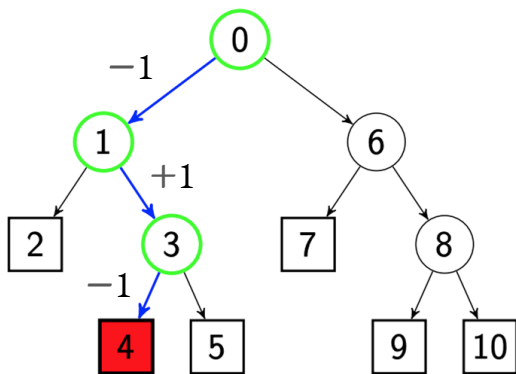
First Hidden Layer output

- By applying the threshold activation function $\tau(h_{k'}(x)) = \tau(x^{(i_{k'})} - \alpha_{i_{k'}})$ each split and relative position of $\mathbf{x}$ in the HDDT is encoded in each neuron, so HL1 outputs a **vector of ± 1 bits**, $(\tau(h_1(\mathbf{x})), \tau(h_2(\mathbf{x})), \dots, \tau(h_{k-1}(\mathbf{x})))$, +1 if $\mathbf{x}$ falls on the right side ($h(\mathbf{x}) \geq 0$), and -1 otherwise.



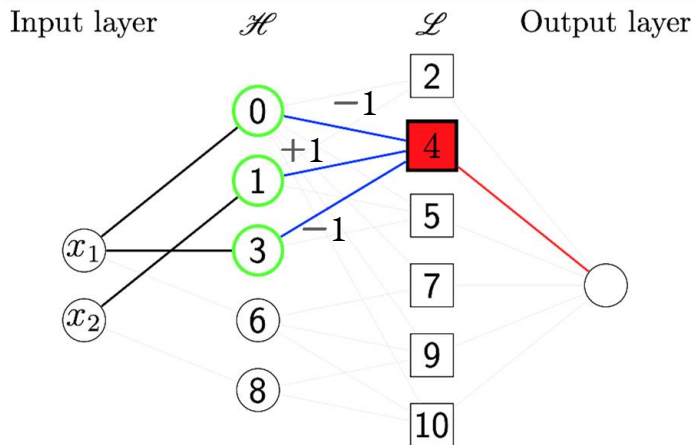
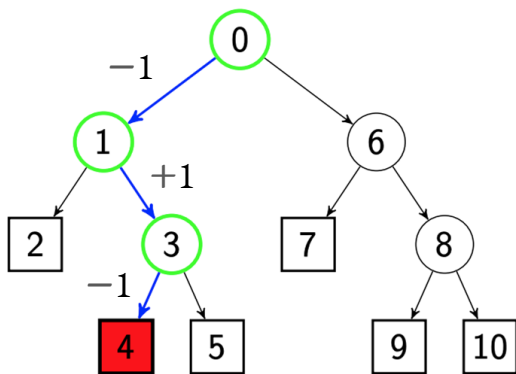
Second Hidden Layer: partition ANDing 1/2

- The second hidden layer (HL2) comprises k leaves, and each connection from k' to k'' has weight $b = +1$ if in order for x to get to k'' , it follows the right branch of k' in HDDT, or $b = -1$ if it follows the left branch.



Second Hidden Layer: partition AND-ing 2/2

- Get $(u_1(x), \dots, u_{k-1}(x))$ from HL1 (a vector of ± 1 bits)
- $$u = \begin{cases} \sum_{k' \rightarrow k''} b_{k',k''} u_{k'}(x) - l(k'') + \frac{1}{2} = \frac{1}{2} & \text{if } x \in L_{k''} \\ \sum_{k' \rightarrow k''} b_{k',k''} u_{k'}(x) - l(k'') + \frac{1}{2} \leq -\frac{1}{2} & \text{if } x \notin L_{k''} \end{cases}$$
- Apply the threshold activation function $\tau(u) \Rightarrow$ “one-hot” vector with -1 s instead of 0s



Output Layer: “one-hot” vector OR-ing

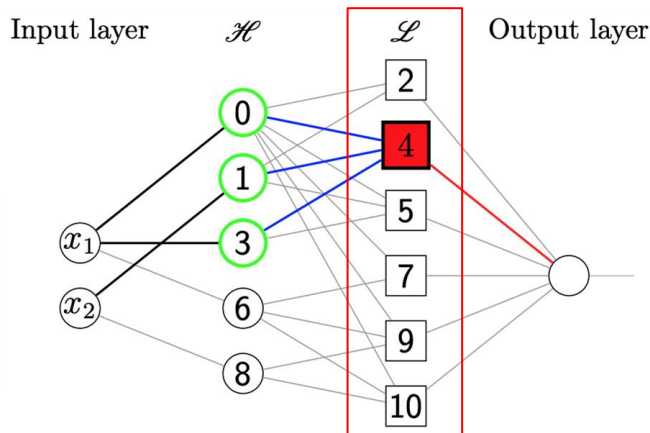
$$t_n(x) = \sum_{k''=1}^k \boxed{w_{k''}} \boxed{v_{k''}(x)} + \boxed{b_{out}} \quad \text{with} \quad \boxed{w_{k''} = \frac{\bar{Y}_{k''}}{2}} \quad \boxed{b_{out} = \frac{1}{2} \sum_{k''=1}^k \bar{Y}_{k''}}$$

$$= \sum_{k''=1}^k v_{k''}(x) \bar{Y}_{k''} \frac{1}{2} + \frac{1}{2} \sum_{k''=1}^k \bar{Y}_{k''}$$

$$= \frac{1}{2} \sum_{k''=1}^k \bar{Y}_{k''} (v_{k''}(x) + 1)$$

$$= \bar{Y}_{k''}$$

$$g_n(x) = \begin{cases} 0, & \text{if } t_n(x) \leq \frac{1}{2} \\ 1, & \text{otherwise} \end{cases}$$

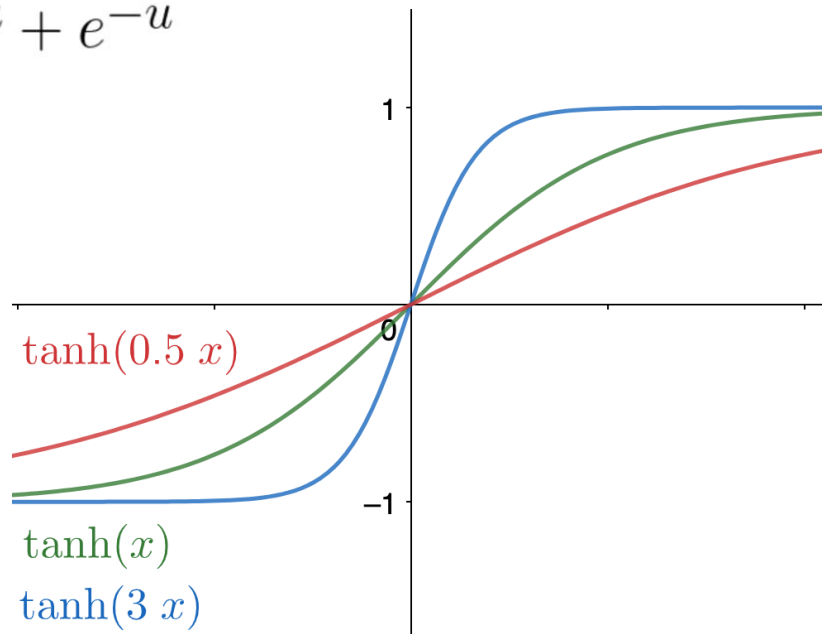


Finishing touches

- The Hellinger net uses a **sigmoid** activation function instead of a threshold, relay-type one

$$\tau(u) = \sigma(u) = \tanh(u) = \frac{e^u - e^{-u}}{e^u + e^{-u}}$$

- HL1 uses $\sigma(\beta_1 u)$, while HL2 uses $\sigma(\beta_2 u)$
- The Hellinger net is trained using **stochastic gradient descent backpropagation** algorithm
- The network requires fitting $O(p \times k + k^2)$ parameters, and if the HDDT is roughly balanced, then $O(k \log(k))$.



Hellinger Net Algorithm

- Build a HDDT with $(k_n - 1)$ split nodes and k_n leaf nodes. HDDT is mapped into a two hidden layered MLP model having $(k_n - 1)$ and k_n hidden neurons in first hidden layer (*HL1*) and second hidden layer (*HL2*), respectively.
- The first hidden layer is called the partitioning layer which partitions the input feature spaces into different regions. It corresponds to the internal nodes of the DT. In *HL1*, the neurons compute all the tree split decisions and indicate the split directions for the inputs.
- Further, *HL1* passes the information to *HL2*. The neurons in the second hidden layer represent the terminal nodes of the DT.
- The final layer is the output class label of the tree. Train the tree structured neural network using gradient descent backpropagation algorithm.
- Hellinger Net uses sigmoidal activation function instead of the relay-type activation function $\tau(u)$ with a hyperbolic tangent activation function $\sigma(u) = \tanh(u)$ which has a chosen range from -1 to 1 .
- More precisely, the model uses $\sigma_1(u) = \sigma(\beta_1 u)$ at every neuron of the first hidden layer for better generalization, where β_1 is a positive hyper-parameter that determines the contrast of the hyperbolic tangent activation function.

- **Merits:**

1. The additional training using backpropagation potentially improves the predictions of the HDDT and can deny tree pruning steps vis-a-vis the risk of overfitting.;
2. Hellinger Nets give weight to nodes according to their significance as determined by the gradient backpropagation algorithm.;
3. In Hellinger Nets, the neural network follows the built-in hierarchy of the originating tree since connections do not exist between all pairs of neurons in any two adjacent layers.;
4. Since the number of neurons in the hidden layers are fixed, thus the training time is less.

- **Theoretical developments:**

1. Theoretical Consistency?
2. Rate of Convergence?

Theorem (Chakraborty et al., 2020, IEEE Transactions on Reliability)

Assume X is uniformly distributed in $[0, 1]^p$ and $Y = \{0, 1\}$. As $n \rightarrow \infty$ and for any $k_n, \beta_1, \beta_2 \rightarrow \infty$ if the following conditions are satisfied:

$$(A1) \quad \frac{k_n^4 \log(\beta_2 k_n^4)}{n} \rightarrow 0,$$

$$(A2) \quad \text{there exists } \delta > 0 \text{ such that } \frac{k_n^2}{n^{1-\delta}} \rightarrow 0,$$

$$(A3) \quad \frac{k_n^2}{e^{2\beta_2}} \rightarrow 0, \quad \text{and}$$

$$(A4) \quad \frac{k_n^3 \beta_2}{\beta_1} \rightarrow 0,$$

then Hellinger Nets classifier is consistent.

The above Theorem states that with certain restrictions imposed on the number k_n of terminal nodes and the parameters β_1, β_2 being properly regulated as functions of n , the empirical L_1 risk-minimization provides local consistency of the Hellinger Nets classifier.

Theorem (Chakraborty et al., 2020, IEEE Transactions on Reliability)

Assume that X is uniformly distributed in $[0, 1]^p$ and $Y = \{0, 1\}$ and a function $m : C^p \rightarrow \{0, 1\}$ is a Lipschitz $(\delta; C)$ -smooth for any $\delta \in [0, 1]$. Let m_n be the estimate that minimizes empirical L_1 -risk and the network activation function σ satisfies Lipschitz property. Then for any $n \geq \max\{\beta_2, 2^{p+1}L\}$, we have

$$E \int_{[0,1]^p} |m_n(X) - m(X)| \mu(dx) = O\left(\frac{\log(n)^6}{n}\right)^{\frac{2}{2+p}}$$

- The proof of the Theorem is using Complexity Regularization Principles.
- The rate of convergence doesn't depend on the data dimension and hence the model will be able to circumvent the so-called problem of "curse of dimensionality".
- In practice, the larger the value of k_n , β_1 , and β_2 , the better the model performance is.

Data Sets: The proposed model is evaluated using five publicly available UCI data sets.

Table: Characteristics of the UCI data sets used in experimental evaluation

Data set	Classes	Objects (n)	Number of feature (p)	Number of (+)ve instances	Number of (−)ve instances	CV
breast cancer	2	286	9	201	85	0.41
german credit card	2	1000	20	700	300	0.40
indian business school	2	480	17	400	80	0.56
page blocks	2	5473	10	4913	560	0.80
pima diabetes	2	768	8	500	268	0.30

The performance evaluation measure used in our experimental analysis is based on the confusion matrix in Table. Area under the receiver operating characteristic curve (AUC) is a popular metric for evaluating performances of imbalanced data sets and higher the value of AUC, the better the classifier is. $AUC = \frac{\text{Sensitivity} + \text{Specificity}}{2}$; where, $\text{Sensitivity} = \frac{TP}{TP+FN}$; $\text{Specificity} = \frac{TN}{FP+TN}$.

Table: AUC results (and their standard deviation) of classification algorithms over original imbalanced test data sets

Classifiers	breast cancer	German credit card	Indian business school	page blocks	pima diabetes
CT	0.603 (0.04)	0.665 (0.03)	0.810 (0.04)	0.950 (0.00)	0.724 (0.02)
RF	0.690 (0.06)	0.725 (0.03)	0.850 (0.04)	0.964 (0.00)	0.747 (0.04)
k-NN	0.651 (0.03)	0.727 (0.01)	0.750 (0.03)	0.902 (0.02)	0.730 (0.05)
RBFN	0.652 (0.06)	0.723 (0.04)	0.884 (0.05)	0.935 (0.01)	0.725 (0.04)
HDDT	0.625 (0.04)	0.738 (0.04)	0.933 (0.02)	0.974 (0.00)	0.760 (0.02)
HDRF	0.636 (0.04)	0.742 (0.03)	0.939 (0.02)	0.988 (0.00)	0.760 (0.03)
ANN (with 1HL)	0.585 (0.03)	0.700 (0.03)	0.768 (0.05)	0.918 (0.02)	0.649 (0.03)
ANN (with 2HL)	0.621 (0.02)	0.715 (0.02)	0.820 (0.04)	0.925 (0.01)	0.710 (0.03)
Hellinger net	0.730 (0.05)	0.802 (0.03)	0.968 (0.01)	0.980 (0.02)	0.809 (0.03)

- The arena of research in learning from imbalanced data” continues to grow, largely driven by challenging problems including land cover classification, fraud detection, face recognition, spam and anomaly detection, medical diagnosis, etc
- The overarching question is “how to push the boundaries of prediction on the underrepresented or minority classes while managing the trade-off with false positives?”
- The usefulness and success of Random Forests and Deep Learning methods are evident. Can they be combined together to create a Deep Forest model that can deal with data imbalance problem?
- Use of Wasserstein Distance is of much use in the Machine Learning community for the last few decades. Some modification to the Wasserstein distance can be done and incorporated in the DT, RF, and Hellinger net model. This may improve the existing HDDT, HDRF and Hellinger Net models for imbalanced pattern classification.

Generalized Wasserstein Distance

Given two CDFs F_1 and F_2 on $\mathbb{R}$, let F denote the set of all joint distribution on $\mathbb{R}^2$ having F_1 and F_2 as marginals.

Definition

Given two CDFs F_1 and F_2 on $\mathbb{R}$, the Wasserstein distance between them is defined by

$$W_1(F_1, F_2) = \left[\inf_{F \in \Gamma(F_1, F_2)} \int_{\mathbb{R}^2} |x - y| dF(x, y) \right] \quad (0.5)$$

Proposed Generalized Wasserstein metric is a linear combination of the Wasserstein distance metric for discrete probability measures (d_E) and the absolute value of the differences in norms parameterized by a real number μ , and is defined as

$$D_\mu(x, y) = d_E \left(\frac{x}{\|x\|}, \frac{y}{\|y\|} \right) + \mu \left| \|x\| - \|y\| \right|$$

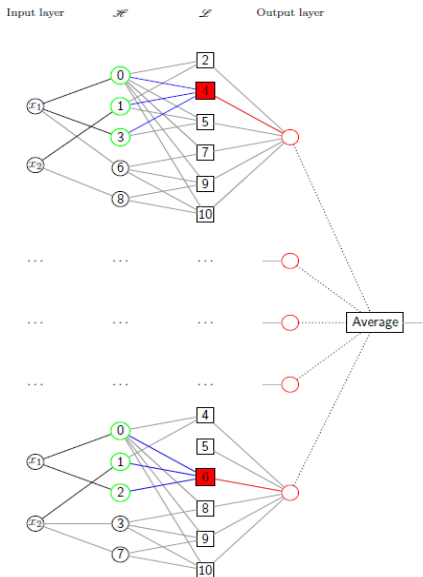
where, $d_E \left(\frac{x}{\|x\|}, \frac{y}{\|y\|} \right) = \sum_{i=1}^p \left| \left(\frac{x_i}{\|x\|} - \frac{y_i}{\|y\|} \right) \right|$. By using the actual norm information in the WD; we hope that the proposed GWD can deal data imbalance and outliers in the data sets better than Hellinger distance and others.

- It is straightforward to see that the proposed generalized Wasserstein distance is a metric.
- When x and y with well distinct norm are far away, we get large μ ; and for small μ their renormalized versions only matters.
- By using a renormalized version of x and y and adding the norm information with weight parameters, we shall be able to make the WD metric skew-insensitive and useful to handle noisy data in imbalanced land cover, aerial imagery and physiological data classification problems.
- Generalized Wasserstein metric will be able to handle the highly imbalanced data problem within the Deep Forest framework.

Deep Forest - Individual Training

Finally, the project aims to make the proposed GWDF model:

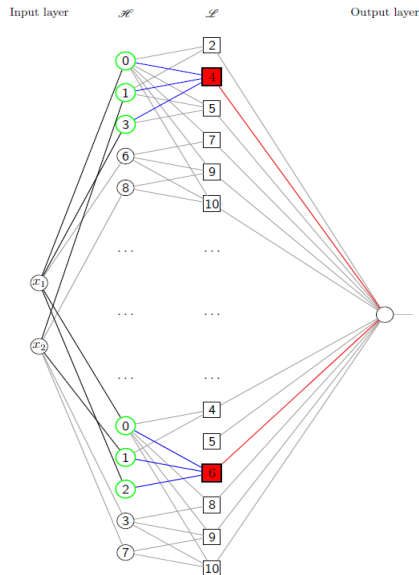
- scalable (the size of the data does not pose a problem),
- robust (work well in a wide variety of problems in the presence of noisy samples),
- accurate (achieve higher predictive accuracy),
- statistically sound (have desired asymptotic properties),
- easily interpretable for its effective implementation in land cover, aerial imagery and physiological data classification.



Deep Forest - Joint Training

We strongly desire that, whilst achieving competitive performance on imbalanced datasets in imbalanced data classification, GWDF would benefit from

- lightweight inference via conditional computation (sparse connected networks),
- hierarchical separation of features useful to the imbalanced learning task with generalized WD metric as tree splitting criteria,
- a mechanism to adapt the architecture to the size and complexity of the training dataset,



- Generalized Wasserstein Deep Forest is a form of random forests enhanced with deep learned representations.
- Many existing tree-structured models are instantiations of the proposed GWDF model.
- The outcome of this work will be a suite of novel, principled, and interpretable deep learning techniques that would solve the imbalanced problem in land cover data classification and others.
- We shall further investigate statistical consistency and rate of convergence for theoretical robustness of the proposed Wasserstein Deep Forest.
- Apart from the theoretical and computational development of the GWDF model and its implementation on real-world datasets, we aim to develop an implementation tool (a Toolbox in Python) for public use.

Thank You!